

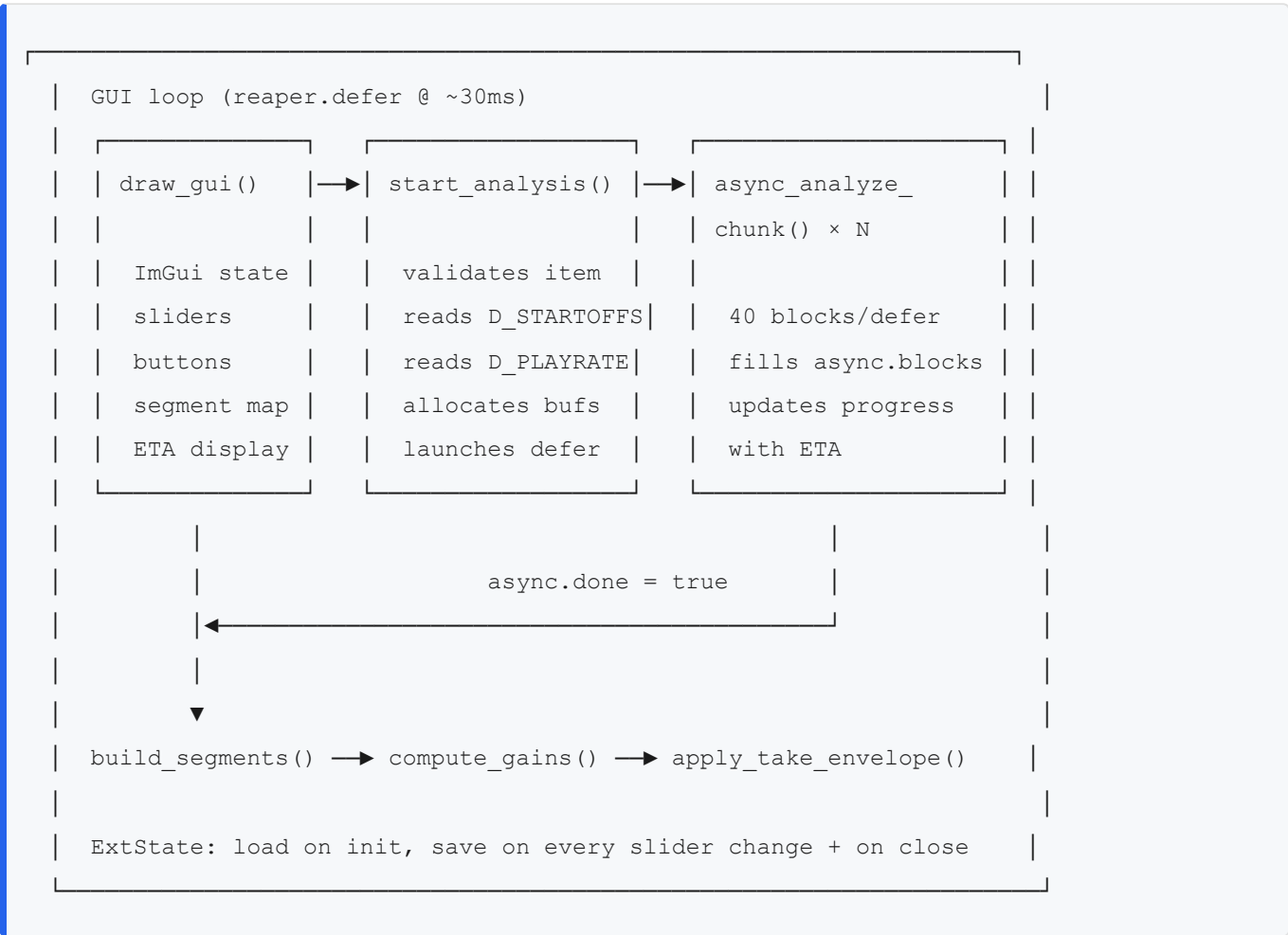
Vocal Volume Macro — Technical Documentation

Internal architecture, DSP algorithms, data flow, and REAPER API notes
Reflects the current production version of `VocalVolumeMacro.lua`

Table of Contents

1. Architecture overview
 2. Configuration table
 3. Signal processing pipeline
 4. Block analysis — `analyze_block`
 5. Async analysis loop
 6. Source time synchronisation
 7. Classification and smoothing
 8. Adaptive silence threshold
 9. Segmentation — `build_segments`
 10. Gain computation — `compute_gains`
 11. Envelope writing — `apply_take_envelope`
 12. Cache system
 13. Settings persistence — `ExtState`
 14. GUI loop and state machine
 15. Known limitations
 16. REAPER API reference
-

1. Architecture overview



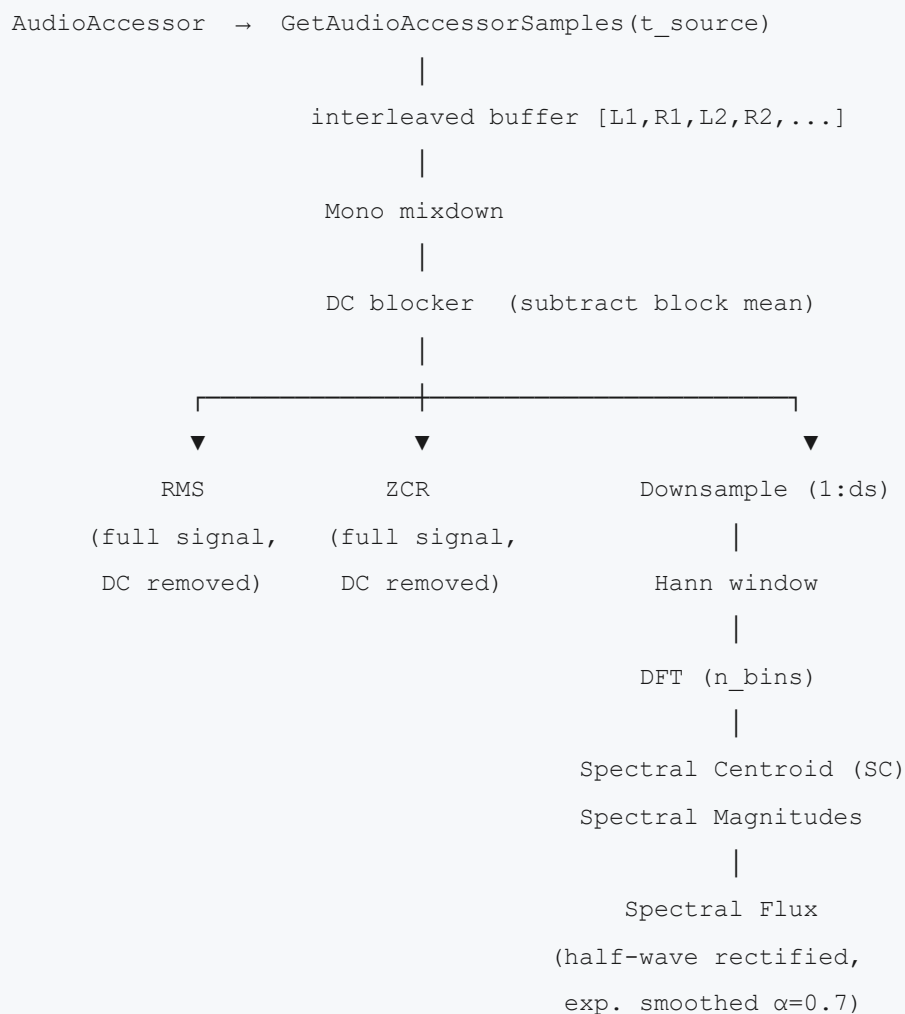
All state lives in three module-level tables:

Table	Purpose
CFG	Static defaults, written from GUI before each analysis
async	Mutable state shared between start_analysis and async_analyze_chunk
gui	GUI state, loaded from ExtState on startup

2. Configuration table

```
local CFG = {  
    block_ms      = 10,      -- block duration (ms)  
    min_silence_db = -50,    -- static floor (overridden by adaptive threshold)  
    zcr_tonal_max  = 0.15,   -- ZCR threshold: tonal if below  
    sc_tonal_max   = 2500,   -- spectral centroid threshold (Hz): tonal if below  
    flux_onset_thresh = 0.30, -- spectral flux threshold: tonal if below  
    smooth_frames  = 3,      -- temporal smoothing window (±N blocks)  
    fade_ms        = 5,      -- transition fade duration (ms)  
    min_seg_ms     = 30,     -- minimum segment duration before merge (ms)  
    dft_downsample = 4,      -- downsampling factor for DFT  
    n_bins         = 50,     -- DFT bins analysed  
    flux_alpha      = 0.7,    -- exponential smoothing for spectral flux  
    per_block_gain  = false,  -- false = segment gain (Melodyne mode), true = Volume R  
    ider  
}
```

3. Signal processing pipeline



Critical ordering: RMS and ZCR are computed on the raw mono signal **before** the Hann window is applied. The Hann window reduces signal energy by ~4.26 dB — computing RMS after windowing would cause the script to systematically underestimate block levels by that amount, corrupting gain targets.

4. Block analysis — `analyze_block`

Signature:

```
local rms, zcr, sc, mags, sum_mag = analyze_block(
  buf, block_samples, nch, samplerate,
  n_bins, ds, N_eff, trig, hann,
  mono, ds_buf, mags -- pre-allocated reusable buffers
)
```

The last three parameters are pre-allocated in `start_analysis` and reused across all blocks. Without pre-allocation, Lua would create and GC ~5400 temporary tables for a 3-minute take.

4.1 Mono mixdown — interleaved buffer

`GetAudioAccessorSamples` fills the buffer in **interleaved** format:

```
buf = { L1, R1, L2, R2, ..., Ln, Rn }
```

Index formula for sample `i`, channel `c` (0-based):

```
buf[(i-1) * nch + c + 1]
```

For mono (`nch == 1`), a direct copy is used.

Critical: Passing a plain Lua table `{}` to `GetAudioAccessorSamples` either returns silence or crashes REAPER with a memory violation. `reaper.new_array(n)` is mandatory.

4.2 DC blocker

```
local dc = sum(mono) / block_samples
if math.abs(dc) > 1e-6 then
    mono[i] = mono[i] - dc -- for all i
end
```

A DC offset shifts the signal baseline. The ZCR counts zero-axis crossings — a shifted signal crosses less often, producing artificially low ZCR that misclassifies noisy blocks as tonal. The `1e-6` threshold skips the correction on perfectly centred signals.

4.3 RMS

```
RMS = sqrt( Σ(mono[i]2) / N )
```

Computed on the full DC-corrected block. Expressed in dB via `lin_to_db()` for classification and gain computation.

4.4 ZCR (Zero Crossing Rate)

```
ZCR = count(sign(mono[i]) ≠ sign(mono[i-1])) / (N - 1)
```

Normalised to [0, 1]. Voiced vowels: ~0.05–0.15. White noise: ~0.5.

4.5 Downsampling + Hann windowing

```
for i = 1, block_samples, ds do
    idx = idx + 1
    ds_buf[idx] = mono[i] * hann[idx]
end
```

Factor `ds = 4` reduces the DFT input from ~441 to ~111 samples. The effective sample rate becomes `sr_eff = samplerate / ds`. The Hann window is applied in the same pass.

N_eff computation — the off-by-one fix:

```
N_eff = floor((block_samples - 1) / ds) + 1
```

This exactly counts the iterations of `for i = 1, N, ds`. The naive `floor(N/ds)` gives N-1 for some values, making the trig tables one element too short and producing `nil` arithmetic at the last sample.

4.6 Hann window

```
w[n] = 0.5 * (1 - cos(2π(n-1)/(N_eff-1)))
```

Reduces spectral leakage at block boundaries. Without windowing, the spectral centroid error is ±15–30%. With Hann: ±2%. Pre-computed once per `N_eff` and cached in `hann_cache`.

4.7 Partial DFT

```
for k = 1, n_bins do
    re = Σ ds_buf[n] * cos(2πk(n-1)/N_eff)
    im = Σ ds_buf[n] * (-sin(2πk(n-1)/N_eff))
    mags[k] = sqrt(re² + im²)
end
```

Only the first `n_bins` (50) of the `N_eff/2` available bins are computed. All `cos` / `sin` values are pre-computed into `trig.cos[k][n]` and `trig.sin[k][n]` tables, cached per `(N_eff, n_bins)`. This avoids ~200 million trig calls for a 3-minute take.

Frequency resolution:

```
freq_step = sr_eff / N_eff
freq_max = n_bins * freq_step
```

4.8 Spectral Centroid

```
SC =  $\Sigma(\text{mags}[k] \times \text{freq}[k]) / \Sigma(\text{mags}[k])$ 
```

Protected against division by zero:

```
local sc = (sum_mag > 0) and (sum_weighted / sum_mag) or 0
```

If `sum_mag` is exactly zero (digital silence from a gate or mute), `sc` returns 0 cleanly instead of producing `NaN` which would propagate through all downstream calculations.

5. Async analysis loop

Chunked execution

```
local BLOCKS_PER_DEFER = 40

local function async_analyze_chunk()
  -- process up to 40 blocks
  -- if not done: reaper.defer(async_analyze_chunk)
  -- if done:      async.done = true
end
```

At 44100 Hz / 10 ms blocks: 40 blocks = 400 ms of audio per defer frame. A 3-minute take (~18 000 blocks) takes ~450 defer cycles $\approx$ ~13 s elapsed. REAPER stays fully responsive throughout.

ETA estimation

The GUI displays a remaining-time estimate based on elapsed time and current progress:

```
local elapsed  = r.time_precise() - gui.analysis_start
local total_est = elapsed / async.progress
local remaining = total_est - elapsed
```

Displayed only after 2% progress to avoid wild early estimates.

pcall safety

The entire chunk body runs inside a `pcall`:

```

local ok, err = pcall(function()
    -- ... all analysis code ...
end)

if not ok then
    if async.aa then r.DestroyAudioAccessor(async.aa); async.aa = nil end
    -- report error to console and GUI
end

```

This catches any unhandled error (nil arithmetic, division by zero, API failure) and ensures the AudioAccessor is always destroyed. Without this, a mid-analysis error leaves the accessor alive, leaking the decoded audio RAM on every subsequent run.

Double-free protection: `async.aa` is set to `nil` immediately after `DestroyAudioAccessor`. The error handler checks `if async.aa` before destroying, preventing a second call on an already-freed object.

6. Source time synchronisation

This is the most important fix for correct behaviour on real-world takes.

The problem

`GetAudioAccessorSamples` reads raw source file data. The time parameter is in **source time**, not item time or project time. If you record 20 seconds and trim the item to show only seconds 5–15, the accessor still has access to the full 20-second source file.

Without synchronisation, `t_start = b * bs / sr` always begins at 0.0 s of the source — the script analyses the hidden trimmed-off section and writes automation that doesn't match the visible content.

The fix — three locations

`start_analysis`: read `D_STARTOFFS` and `D_PLAYRATE` from the take, inject into `async`:

```

local start_offs = r.GetMediaItemTakeInfo_Value(take, "D_STARTOFFS")
local playrate   = r.GetMediaItemTakeInfo_Value(take, "D_PLAYRATE")
if playrate == 0 then playrate = 1.0 end

async.start_offs = start_offs
async.playrate   = playrate

```


Property	Meaning
<code>D_STARTOFFS</code>	Seconds from file start to the beginning of the visible item
<code>D_PLAYRATE</code>	Speed factor (1.0 = normal, 0.5 = half speed, 2.0 = double)

`async_analyze_chunk`: compute `t_source` and pass it to `GetAudioAccessorSamples`:

```
local t_proj    = b * bs / sr                -- time relative to item start
  local t_source = start_offs + (t_proj * playrate)  -- position in source file

  r.GetAudioAccessorSamples(aa, sr, nch, t_source, bs, buf)

blocks[b + 1] = {
  time = t_source,  -- stored directly — ready for InsertEnvelopePoint
  ...
}
```

`apply_take_envelope`: `blocks[i].time` already contains the correct source coordinate — used directly in `InsertEnvelopePoint` without any further conversion:

```
local env_t = blocks[i].time  -- already t_source, no recalculation needed
  r.InsertEnvelopePoint(env, env_t, to_env(g_cur), 0, 0, false, true)
```

The Take Volume Envelope API expects times in source space, which is exactly what `t_source` provides.

7. Classification and smoothing

Raw classification

A block is tonal only if **all four** conditions hold simultaneously:

```
audible  = rms_db > adaptive_silence      -- above noise floor
  low_zcr = zcr    < cfg.zcr_tonal_max    -- periodic signal
  low_sc  = sc     < cfg.sc_tonal_max     -- energy in low frequencies
  no_onset = flux  < cfg.flux_onset_thresh -- no abrupt spectral attack
```

Spectral flux

```
flux = 0
for k = 1, n_bins do
  diff = mags[k] - prev_mags[k]
  if diff > 0 then flux = flux + diff end  -- half-wave rectification
end
flux = flux / sum_mag  -- normalise by total energy

flux_smooth = alpha * flux_smooth + (1 - alpha) * flux  --  $\alpha = 0.7$ 
```

Half-wave rectification captures energy increases only — characteristic of consonant onsets. The exponential smoother prevents single-frame spikes from triggering false detections.

Temporal smoothing — majority vote

```
for i = 1, #blocks do
  votes = 0
  for j = max(1, i - sf), min(N, i + sf) do
    votes += raw_tonal[j] ? 1 : 0
  end
  blocks[i].is_tonal = (votes / window_size) >= 0.5
end
```

Window: ± 3 blocks (70 ms at 10 ms/block). Prevents isolated outlier blocks from creating spurious 10 ms segments.

8. Adaptive silence threshold

After all blocks are analysed, the noise floor is estimated from the recording itself:

```
table.sort(rms_values)
local p10_idx      = max(1, floor(N * 0.10))
local noise_rms    = rms_values[p10_idx]
local noise_db     = lin_to_db(noise_rms)
local adaptive_silence = clamp(noise_db + 6, -60, -30)
```

Why percentile 10: represents the quietest 10% of blocks — room noise, breath gaps, pre-roll silence. Using the minimum would be fragile against a few anomalous blocks.

+6 dB margin: blocks only slightly above the noise floor (residual breath noise) are excluded from tonal classification.

Clamped to [-60, -30]: prevents unreasonable thresholds in extreme cases (near-silent recording or very noisy environment).

The computed threshold is logged to the REAPER Console:

```
[VVM] Noise floor estimate: -52.3 dB → adaptive silence threshold: -46.3 dB
```

9. Segmentation — `build_segments`

Pass 1 — raw grouping

Consecutive blocks of the same type are grouped into segments. Each segment stores: `from`, `to`, `is_tonal`, `avg_rms` (weighted), `count`.

Pass 2 — short segment merging (backward iteration)

```
for i = #raw, 1, -1 do      -- BACKWARD: table.remove(i) shifts indices > i only
  if raw[i].count < min_blks then
    -- Priority 1: neighbour of same is_tonal type (preserves semantics)
    -- Priority 2: longer neighbour (minimises distortion)
    merge into target
    table.remove(raw, i)
  end
end
end
```

Why backward: `table.remove(raw, i)` shifts all elements at indices `> i` down by one. Iterating forward causes the loop to skip the element that moved into position `i`. Iterating backward avoids this because all processed indices are already `> i` and have been handled.

After merging, `table.sort` restores chronological order.

10. Gain computation — `compute_gains`

Signature:

```
local gains = compute_gains(segments, blocks, target_db, macro_pct, per_block, smooth_alpha)
```

`smooth_alpha` defaults to `0.25` if not provided.

Mode: segment gain (Melodyne behaviour — default)

A single constant gain is applied across the entire segment, based on its average RMS:

```
local applied = (target_db - seg_avg_rms_db) * macro_pct
local gain_lin = clamp(db_to_lin(applied), gain_min, gain_max)
for i = seg.from, seg.to do gains[i] = gain_lin end
```

This preserves the natural internal dynamics of each syllable — the consonant attack and the vowel decay remain intact. This is equivalent to Melodyne's blob volume adjustment.

Mode: per-block gain (Volume Rider)

Each tonal block receives its own gain based on its individual RMS:

```
for i = seg.from, seg.to do
  if blocks[i].rms_db > -60 then
    local applied = (target_db - blocks[i].rms_db) * macro_pct
    gains[i] = clamp(db_to_lin(applied), gain_min, gain_max)
  end
end
```

Blocks below -60 dB keep `gains[i] = 1.0` (near-silence left untouched).

Clamp bounds

Pre-computed outside all loops:

```
local gain_min = db_to_lin(-18)  -- prevents extreme attenuation
local gain_max = db_to_lin(12)   -- prevents extreme amplification and noise boostin
g
```

Anti-zipper double-pass filter (Volume Rider mode only)

Without smoothing, adjacent blocks with different gains produce abrupt gain jumps (zipper noise — audible distortion at 10 ms boundaries). The double-pass filter eliminates this:

```

-- Forward pass (left → right): smooths but introduces phase delay
local cur_g = gains[seg.from]
for i = seg.from + 1, seg.to do
    cur_g = cur_g + alpha * (gains[i] - cur_g)
    gains[i] = cur_g
end

-- Backward pass (right → left): cancels the phase delay
cur_g = gains[seg.to]
for i = seg.to - 1, seg.from, -1 do
    cur_g = cur_g + alpha * (gains[i] - cur_g)
    gains[i] = cur_g
end

```

Zero-phase result: the forward pass introduces group delay $-\phi(\omega)$; the backward pass introduces $-\phi(-\omega) = +\phi(\omega)$, exactly cancelling it. The gain envelope is centred on the syllable rather than trailing it.

Segment-restricted smoothing: the filter runs independently on each tonal segment. Non-tonal blocks (`gains[i] = 1.0`) are never touched. Without this restriction, the gain from a loud syllable would leak into adjacent silence, boosting breath noise.

alpha parameter (0.05–0.60, default 0.25, user-adjustable): controls the smoothing bandwidth. At $\alpha = 0.25$, a step change spreads over ~4–5 blocks (40–50 ms). Higher $\alpha \rightarrow$ faster response, more zipper risk.

11. Envelope writing — `apply_take_envelope`

Take Envelope vs Track Envelope

The automation is written to the **Take Volume Envelope**, not the track envelope. The take envelope is stored inside the item — it moves with the item if you reposition it on the timeline. Track envelope automation does not follow item moves.

Envelope scaling mode

```

local env_mode = r.GetEnvelopeScalingMode(env)

local function to_env(g)
    return r.ScaleToEnvelopeMode(env_mode, g)
end

```

REAPER supports two envelope modes: amplitude (linear) and volume fader (dB). Inserting a raw linear value without conversion produces incorrect results in volume-fader mode, and silently clips at the project's envelope ceiling (typically +6 dB = ~1.995 linear). `ScaleToEnvelopeMode` converts correctly for whichever mode is active.

Intelligent decimation

Without decimation: 18 000 points for a 3-minute take — visually dense, impossible to edit manually.

Two complementary thresholds decide whether to insert a point:

Condition	Threshold	Rationale
<code>is_step</code>	<code>> 0.05</code> linear (≈ 0.4 dB)	Abrupt tonal→silence boundary
<code>is_drift</code>	<code>> 0.015</code> linear (≈ 0.1 dB)	Slow progressive change within a segment
<code>is_last</code>	always	Ensures the final block is always anchored

Result: typically a few hundred points per take. The count is logged:

```
[VVM] 312 envelope points inserted (decimation active).
```

Fade ramps at transitions

When `is_step` is true, a pre-transition fade point is inserted to avoid clicks:

```
local fade_start = math.max(
    env_t - (actual_fade * playrate),
    env_t_prev + (block_sec * 0.1 * playrate)
)
```

Fade clamp: `actual_fade = min(fade_sec, block_sec * 0.9)`. Prevents `fade_start` from being placed before `env_t_prev`, which would create a point with an earlier timestamp than the preceding point — invalid in REAPER's envelope system.

Second guard: `env_t_prev + block_sec * 0.1 * playrate` — ensures at least 10% of a block gap between consecutive points regardless of the fade setting.

Cleanup and finalisation

```
r.DeleteEnvelopePointRange(env, 0, 1e12) -- clear all existing points
-- ... insert all points with noSort=true ...
r.Envelope_SortPoints(env)                -- sort once at the end
r.UpdateItemInProject(item)
r.UpdateArrange()
```

`noSort = true` on every `InsertEnvelopePoint` defers REAPER's internal sort until all points are written. Without this, REAPER re-sorts after every insertion — $O(N^2)$ for large point counts. `Envelope_SortPoints` is mandatory after mass insertion with `noSort = true`. If a fade point was calculated slightly before its predecessor (floating-point rounding), REAPER's internal ordering would be corrupt, causing visual artefacts and playback glitches.

12. Cache system

Trig table cache

```
local trig_cache = {}
  -- key: "N_eff_n_bins" (e.g. "111_50")

local function get_trig_tables(N_eff, n_bins)
  local key = N_eff .. "_" .. n_bins
  if trig_cache[key] then return trig_cache[key] end
  -- compute cos[k][n] and sin[k][n] for all k ∈ [1,n_bins], n ∈ [1,N_eff]
end
```

Pre-computing all trigonometric values eliminates ~200 million `math.cos` / `math.sin` calls for a 3-minute take.

Hann window cache

```
local hann_cache = {}
  -- key: N_eff (integer)
```

Identical principle — the window is the same for every block of the same size.

Cache invalidation

```
local function purge_caches()
  trig_cache = {}
  hann_cache = {}
end
```

Called in the GUI when `block_ms` changes between analyses. Without purging, stale tables from a different `N_eff` produce wrong DFT results silently. Lua's GC frees the old tables automatically once no references remain.

13. Settings persistence — ExtState

All GUI parameters are persisted across REAPER sessions using `reaper.GetExtState` / `reaper.SetExtState`:

```
local EXT_SECTION = "VocalVolumeMacro"

local function ext_load(key, default)
    local s = r.GetExtState(EXT_SECTION, key)
    return (s ~= "") and tonumber(s) or default
end

local function ext_save(key, val)
    r.SetExtState(EXT_SECTION, key, tostring(val), true) -- true = persist to disk
end
```

Persisted parameters: `zcr_thresh`, `sc_thresh`, `flux_thresh`, `block_ms`, `target_db`, `macro_pct`, `fade_ms`, `per_block`, `smooth_alpha`.

Save triggers:

- Every slider/checkbox change in the GUI
- Window close

Load trigger: once at startup, before the `gui` table is initialised.

`ExtState` is stored in REAPER's `reaper.ini` (Windows) or equivalent on macOS/Linux. Values survive REAPER restarts. The section key `"VocalVolumeMacro"` namespaces the values to prevent collisions with other scripts.

14. GUI loop and state machine

Window initialisation

```
r.ImGui_SetNextWindowSize(ctx, W, H, r.ImGui_Cond_FirstUseEver())
r.ImGui_SetNextWindowPos(ctx, 80, 60, r.ImGui_Cond_FirstUseEver())
```

`Cond_FirstUseEver` applies the default size and position only on the first ever launch. On subsequent launches, ImGui restores the user's last position and size. `Cond_Once` (the previous value) could prevent the user from resizing the window on some OS/ReaImGui version combinations.

Defer loop

```
local function loop()
  local open = draw_gui()
  if open then
    r.defer(loop)
  else
    save_settings(gui)
    if async.running then async.cancelled = true end
    r.ImGui_DestroyContext(ctx)
  end
end
```

Closing the window triggers a final `save_settings` call and cancels any in-progress analysis before destroying the ImGui context.

GUI state machine

```
IDLE
|  select item in REAPER
▼
ITEM_SELECTED
|  click Analyse → locks item, stores analysis_start timestamp
▼
ANALYSING — cancel → IDLE (flux_smooth + prev_mags reset)
|
|  async.done = true (pcall succeeded)
▼
ANALYSED
|  click Apply → guard: checks segments not empty
▼
APPLIED  (Ctrl+Z in REAPER undoes the envelope write)
|  click Reset env. → clears Take Envelope, writes neutral 1.0 point
|  click Change      → unlocks item, clears analysis
▼
IDLE
```

`block_ms` invalidation

Changing `block_ms` resets the analysis:

```
if changed then
    gui.analyzed      = false
    gui.blocks        = nil
    gui.segments      = nil
    gui.preview_gains = nil
end
```

This prevents Apply from using blocks computed at a different resolution than the current setting. Other analysis parameters (ZCR, SC, Flux) require a re-analysis to take effect but do not invalidate existing results — the user can still apply the previous analysis with updated classification thresholds.

15. Known limitations

Timestretched takes (REAPER's built-in stretch)

`GetAudioAccessorSamples` reads the raw source file, not the time-stretched audio. The `D_PLAYRATE` sync fix handles constant-rate playrate changes correctly. However, REAPER's élastique/SoundTouch time-stretch (where pitch is preserved during tempo change) operates differently — the source file is unchanged and the stretching is applied in real time during playback. The accessor reads unstretched audio, so analysis will be on the original tempo, but the envelope will be applied on the stretched playback. Render the item to a new file before running the script if élastique stretch is active.

Very short takes

At `block_ms = 10`, a 50 ms take produces only 5 blocks. The double-pass smoothing across 5 values produces nearly identical gains throughout. Per-block mode effectively degrades to segment mode in this case.

Envelope ceiling

`ScaleToEnvelopeMode` respects REAPER's configured envelope ceiling (typically +6 dB = ~ 1.995 linear). Gains above this are silently clamped by REAPER regardless of what the script inserts. The script's own clamp of `db_to_lin(12)` = ~ 3.98 can exceed the project ceiling, so REAPER's ceiling may be the effective upper limit.

n_bins and low sample rates

At 22050 Hz with `ds = 4`:

```
sr_eff    = 5512 Hz
N_eff     = 56
freq_res  = 98 Hz
freq_max  = 50 × 98 = 4900 Hz
```

At 11025 Hz, `freq_max` drops further. The SC slider upper bound is automatically adjusted to `freq_max` after analysis, but classification precision decreases. Use `block_ms = 20` or `ds = 2` at low sample rates.

16. REAPER API reference

API call	Notes
<code>r.new_array(n)</code>	Mandatory for <code>GetAudioAccessorSamples</code> . Lua <code>{}</code> crashes or returns silence.
<code>r.GetAudioAccessorSamples(aa, sr, nch, t, bs, buf)</code>	<code>t</code> is in source time (seconds from file start). Buffer is interleaved: <code>buf[(i-1)*nch+c+1]</code> .
<code>r.CreateTakeAudioAccessor(take)</code>	Decodes and buffers audio in RAM. Must be destroyed with <code>r.DestroyAudioAccessor()</code> .
<code>r.DestroyAudioAccessor(aa)</code>	Set <code>async.aa = nil</code> immediately after to prevent double-free.
<code>r.GetMediaItemTakeInfo_Value(take, "D_STARTOFFS")</code>	Seconds from source file start to the beginning of the visible item.
<code>r.GetMediaItemTakeInfo_Value(take, "D_PLAYRATE")</code>	Take playback rate. 1.0 = normal. Guard against 0 before using as divisor.
<code>r.GetTakeEnvelopeByName(take, "Volume")</code>	Returns <code>nil</code> if the take volume envelope has not been created yet.
<code>r.Main_OnCommand(40693, 0)</code>	Toggles take volume envelope existence. Action ID may vary by REAPER version.
<code>r.GetEnvelopeScalingMode(env)</code>	Returns 0 (amplitude) or 1 (volume fader dB). Required before <code>ScaleToEnvelopeMode</code> .
<code>r.ScaleToEnvelopeMode(mode, g)</code>	Converts a linear gain to the correct internal envelope value for the active mode.
<code>r.DeleteEnvelopePointRange(env, 0, 1e12)</code>	Clears entire take envelope.
<code>r.InsertEnvelopePoint(env, t, v, 0, 0, false, true)</code>	Last arg <code>noSort=true</code> — defer sorting until all points are inserted.
<code>r.Envelope_SortPoints(env)</code>	Required after mass insertion with <code>noSort=true</code> .
<code>r.GetExtState(section, key)</code>	Returns empty string if key not set — always check <code>s ~= ""</code> before using.
<code>r.SetExtState(section, key, val, true)</code>	<code>true</code> = persist to disk across REAPER restarts.
<code>r.time_precise()</code>	High-resolution timer for ETA calculation. Not affected by REAPER's transport.
<code>r.defer(fn)</code>	Schedules <code>fn</code> on next UI frame (~30 ms). Used for async analysis and GUI loop.
<code>r.ImGui_Cond_FirstUseEver()</code>	Apply window size/pos only on first ever launch. Preferred over <code>Cond_Once</code> for resizable windows.

ReaImGui version note

The script uses `r.ImGui_BeginTable` / `r.ImGui_EndTable` (not the deprecated `r.ImGui_Columns`).
The table API requires ReaImGui 0.7 or later. If you see errors about `ImGui_BeginTable` , update ReaImGui via ReaPack.